

AI-Powered Appliance Pricing Intelligence Platform

1. Business Problem

Consumers often struggle to determine whether an appliance listed on Amazon, Flipkart, or another marketplace is fairly priced. The goal is not merely to predict prices. The goal is to help consumers make better purchasing decisions. The platform will:

- Estimate the fair market value of an appliance using machine learning.
- Compare fair value against the current listed price.
- Determine whether the appliance is fairly priced, overpriced, or a good value.
- Suggest similar alternatives with better value-for-money.
- Explain which features contribute most to the predicted value.

Final user outcome: 'Should I buy this appliance at this price?'

2. Coding Environment

Recommended stack: Language: - Python 3.11+ Core Libraries: - pandas - numpy - scikit-learn - xgboost - matplotlib - seaborn - shap - joblib Data Collection: - requests - beautifulsoup4 - selenium (optional) Backend: - FastAPI Frontend: - Next.js or React Storage: - CSV initially - PostgreSQL later Deployment: - Vercel (frontend) - Render (backend)

3. Data Requirements

Primary source: Smartprix appliance listings. Required fields: - Product Name - Brand - Category - Current Price - Energy Rating - Capacity - Inverter/Non-Inverter - Warranty - Customer Rating - Review Count - Additional Specifications Target Variable: - Current Market Price Features: - Brand - Category - Capacity - Energy Rating - Warranty - Rating - Technical specifications

4. Models Used

Baseline: - Linear Regression Regularized Models: - Ridge Regression - Lasso Regression Tree-Based Models: - Decision Tree Regressor - Random Forest Regressor Advanced Model: - XGBoost Regressor Interpretability: - Feature Importance - SHAP Recommendation Layer: - Similarity search using appliance specifications.

5. End-to-End Workflow

Step 1: Scrape appliance data from Smartprix. Step 2: Clean and preprocess data. Step 3: Perform exploratory data analysis. Step 4: Engineer useful features. Step 5: Train Linear Regression baseline. Step 6: Train Ridge and Lasso models. Step 7: Train Random Forest and XGBoost. Step 8: Compare models using RMSE, MAE, and R^2 . Step 9: Save the best model. Step 10: Build pricing intelligence engine. User Input: - Product specifications - Current marketplace price Model Output: - Estimated Fair Price Decision Engine: - Compare Fair Price vs Actual Price - Compute Premium/Discount Percentage - Generate Value Score Alternative Finder: - Find similar products with lower prices. Explainability: - Show factors driving predicted value. Frontend: - User submits appliance details. - Dashboard displays fair value, verdict, and alternatives. Deployment: - Deploy model API and frontend.